FRAUD DETECTION USING GRAPH NEURAL NETWORKS

A Complete Technical Thesis

Elliptic Bitcoin Dataset · IEEE-CIS Fraud Detection

Data Transformation · Graph Construction · GNN Architecture · Training · Prediction

Abstract

This thesis presents a complete technical treatment of fraud detection using Graph Neural Networks (GNNs), applied to two real-world financial datasets: the Elliptic Bitcoin Dataset and the IEEE-CIS Fraud Detection Dataset. Financial fraud is a fundamentally relational phenomenon — fraudulent entities form interconnected networks of accounts, transactions, and shared identities that are invisible to standard tabular machine learning.

We model payment networks as graphs and apply two GNN architectures: GraphSAGE (Hamilton et al., 2017) with scalable mean aggregation, and Graph Attention Networks (GAT, Velickovic et al., 2018) with adaptive attention weights. Three complete training experiments on the Elliptic dataset (203,769 nodes, 2.23% fraud) demonstrate that GraphSAGE with weighted loss achieves Test F1=0.907, AUC-ROC=0.987, and AUC-PR=0.953 — a 43× improvement over the random AUC-PR baseline of 0.022.

Metric	GraphSAGE + Weighted Loss	GAT + Weighted Loss	GraphSAGE + Both Strategies
Test F1	0.9073 ★	0.8453	0.8110
Precision	0.9175	0.8898	0.7609
Recall	0.8974	0.8050	0.8680
AUC-ROC	0.9870	0.9817	0.9739
AUC-PR	0.9534	0.9149	0.8963
Epochs	150	86 (early stop)	113 (early stop)

1. Problem Statement

1.1 The Financial Fraud Landscape

Financial fraud causes hundreds of billions in annual losses. As payments went digital, fraudsters evolved from simple card skimming to coordinated multi-entity network attacks.

Fraud Type	Mechanism	Scale	Detection Challenge
Card Fraud	Stolen credentials for unauthorized transactions	$32B/year	High transaction velocity; feature overlap with legitimate use
Account Takeover	Hijack legitimate account credentials	$11B/year	Mimics legitimate behavior — feature separation is difficult
Money Laundering	Layer illicit funds through legitimate accounts	$2T/year	Multi-hop obfuscation; single-transaction view misses the pattern
Synthetic Identity	Combine real and fictitious data to create fake identities	$20B/year	No historical fraud signal — only structural indicators available
Crypto Crime	Illicit Bitcoin/Ethereum transaction flows	$10B/year	Pseudonymous; requires full graph-level flow analysis

1.2 Why Standard ML Fails

Conventional ML models treat each transaction as an independent, identically distributed sample. This is violated in four structural ways:

- **Extreme Class Imbalance:** 2.23% fraud on Elliptic (as low as 0.01% in production). A 'predict legit for everything' classifier achieves 97.77% accuracy — useless. F1, AUC-PR, and Precision/Recall are required.
- **Relational Structure Is the Signal:** Fraudsters form mule networks — rings of accounts passing funds to obscure origin. The signal is network topology, not individual transaction features. Tabular models structurally cannot see this.
- **Adversarial Adaptation:** Fraud patterns evolve as detection improves. Models must generalize to unseen topological signatures, not memorize feature patterns.
- **Label Scarcity:** 77.2% of Elliptic nodes are unlabelled. In production, ground truth arrives weeks late via chargebacks. Models must learn from partial labels.

> **!** *Core insight: Fraud is a relational phenomenon. A transaction is not fraudulent in isolation — it is fraudulent because of who sent it, who received it, and the network topology connecting them. GNNs are relational models built for exactly this.*

2. Datasets

2.1 Elliptic Bitcoin Dataset

A directed graph of Bitcoin transactions collected 2011–2019, spanning 49 time steps (~2 weeks each). Primary benchmark for GNN-based fraud detection research.

Property	Value	Significance
Total nodes	203,769 transactions	Large-scale; requires mini-batch training
Total edges	234,355 directed UTXO flows	Very sparse: average degree = 1.15
Labelled nodes	46,564 (22.8%)	77.2% unknown — severe label scarcity
Illicit (fraud)	4,545 (2.23% of all nodes)	Extreme class imbalance
Licit (legit)	42,019 (20.6%)	Clear majority class
Unknown nodes	157,205 (77.2%)	Model must generalize to unlabelled nodes
Features per node	166 (94 local + 72 aggregated)	94 transaction-level + 72 neighborhood statistics
Time steps	49 chronological	Enables temporal evaluation experiments
Max node degree	472	Hub nodes exist; most nodes have degree 1–2

2.1.1 Raw File Structure

File	Columns	Role
elliptic_txs_features.csv	txId, time_step, feat_0 … feat_165	Node feature matrix — 166 features per transaction
elliptic_txs_edgelist.csv	txId1, txId2	Directed edges: output of txId1 consumed as input to txId2
elliptic_txs_classes.csv	txId, class (1=illicit / 2=licit / unknown)	Node labels

2.1.2 Graph Semantics

Each node is a Bitcoin transaction (not a wallet). An edge A→B means the UTXO output of A was consumed as input to B. The 94 local features encode amount, fee, input/output count, and time step. The 72 aggregated features are 1-hop neighborhood statistics computed by Elliptic.

2.2 IEEE-CIS Fraud Detection Dataset

An e-commerce fraud dataset from Vesta Corporation. Unlike Elliptic, no graph is provided — it must be constructed from the tabular data via card entity resolution.

Property	Value	Significance
Transactions	590,540 rows	Larger than Elliptic
Fraud rate	3.5%	Still severe imbalance
Transaction features	V1–V339 (Vesta-engineered)	Rich behavioral features from payment processor
Identity features	id_01 to id_38	Device, browser, and network identity signals
Graph type	Bipartite: Cards ↔ Transactions	Constructed — not given explicitly
Key challenge	No explicit graph structure	Must build edges from entity resolution

2.2.1 Bipartite Construction

A composite key card1 + '_' + card2 + '_' + card4 identifies each unique card. Card nodes and transaction nodes form two sets; edges connect each card to every transaction it made.

```
Card nodes:  IDs 0 … (n_cards-1)        features: zeros (no card-level features)
Txn nodes:   IDs n_cards … (n_cards+n_txns-1)  features: numeric columns

Edge:  card_ABC ──[made]──> txn_12345
       card_ABC ──[made]──> txn_67890   (same card, multiple transactions)

Undirected: edge_index = concat([card→txn], [txn→card])
Labels: card nodes y=-1,  txn nodes y=0 (legit) or y=1 (fraud)
```

2.3 Dataset Comparison

Dimension	Elliptic Bitcoin	IEEE-CIS
Node meaning	1 Bitcoin transaction	Credit card OR transaction (two types)
Edge meaning	UTXO flow: output of A → input of B	Card made this transaction
Graph type	Homogeneous, directed	Bipartite, undirected
Graph source	Provided explicitly	Constructed by entity resolution
Node count	203,769	~1.2M (cards + transactions)
Average degree	1.15 (extremely sparse)	Varies by card activity
Label coverage	22.8% labelled	100% of transaction nodes
Temporal info	49 time steps given	No temporal structure
Primary use	Main experiments	Pipeline validation

3. Data Transformation Pipeline

Both datasets are transformed from raw CSV files into PyTorch Geometric Data objects. The two pipelines differ in graph construction but produce the same output format.

3.1 Elliptic Transformation Pipeline

Figure 1 — Elliptic Bitcoin Dataset: Raw CSV → PyG Data Object

Boolean mask tensors shape [203769]: train_mask / val_mask / test_mask

↓ **Output** ↓

OUTPUT — PyG Data Object

data.x = FloatTensor [203769, 166] node features
data.edge_index = LongTensor [2, 234355] directed edges
data.y = LongTensor [203769] labels (0/1/-1)
data.train_mask = BoolTensor [203769] training nodes
data.val_mask = BoolTensor [203769] validation nodes
data.test_mask = BoolTensor [203769] test nodes

3.2 IEEE-CIS Transformation Pipeline

Figure 2 — IEEE-CIS Fraud Detection: Tabular CSV → Bipartite Graph

RAW INPUT: 2 CSV Files

train_transaction.csv [590,540 × 394]
train_identity.csv [144,233 × 41]

↓ **Step 1: Identity Join** ↓

STEP 1 — Left Join on TransactionID

txn_df = train_transaction.merge(train_identity, on='TransactionID', how='left')
Result: [590,540 × 433]
Unmatched transactions get NaN identity features

↓ **Step 2: Card Entity Resolution** ↓

STEP 2 — Composite Card Key

card_id = str(card1) + '_' + str(card2) + '_' + str(card4)
Identifies unique physical credit card across transactions
unique_cards ≈ 500K entities
card_to_idx = { 'card_ABC': 0, 'card_DEF': 1, … }

↓ **Step 3: Bipartite Node Assignment** ↓

CARD NODES (Type 0)

IDs: 0 … (n_cards-1)
Features: zero matrix
Labels: y=-1 (unknown)
~500K card entities

TRANSACTION NODES (Type 1)

IDs: n_cards … (n_cards+590540-1)
Features: numeric columns
Labels: isFraud (0 or 1)
590,540 transactions

↓ **Step 4: Bipartite Edge Construction** ↓

STEP 4 — Edge Index (Undirected)

card_node_id = card_to_idx[card_id column] shape [590540]
txn_node_id = n_cards + arange(590540) shape [590540]
Undirected: concat [card→txn] + [txn→card] directions
edge_index shape [2, ~1.18M edges]

↓ **Output** ↓

OUTPUT — Bipartite PyG Data Object

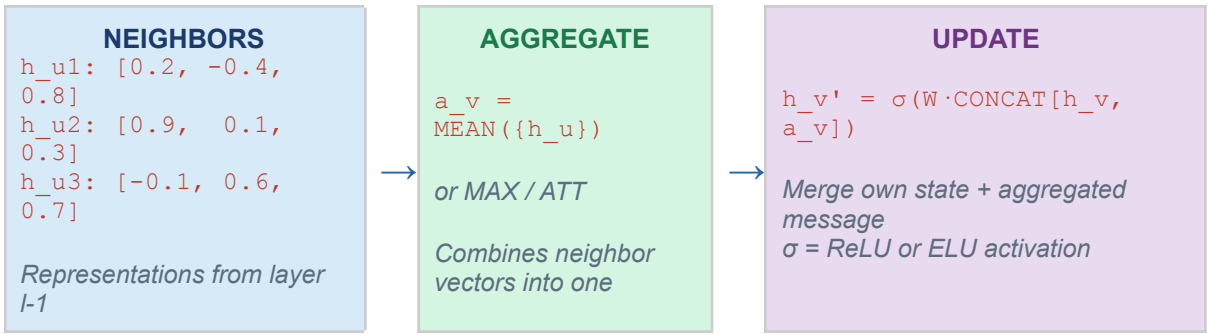
data.x = FloatTensor [n_cards+590540, n_features]
data.edge_index = LongTensor [2, ~1.18M]
data.y = LongTensor [n_cards+590540] (cards=-1, txns=0/1)
Masks cover transaction nodes only

4. Graph Neural Network Architecture

4.1 Message Passing Framework

All GNNs here use the Message Passing Neural Network (MPNN) framework. Every node updates its representation by aggregating from its neighbors. After L layers, each node encodes its L-hop neighborhood.

Figure 3 — MPNN: Single Layer Message Passing




```
Step 1 — Aggregate:   a_v^(l) = AGGREGATE^(l) ({ h_u^(l-1) : u in N(v) })
Step 2 — Update:      h_v^(l) = UPDATE^(l) ( h_v^(l-1),  a_v^(l) )
```

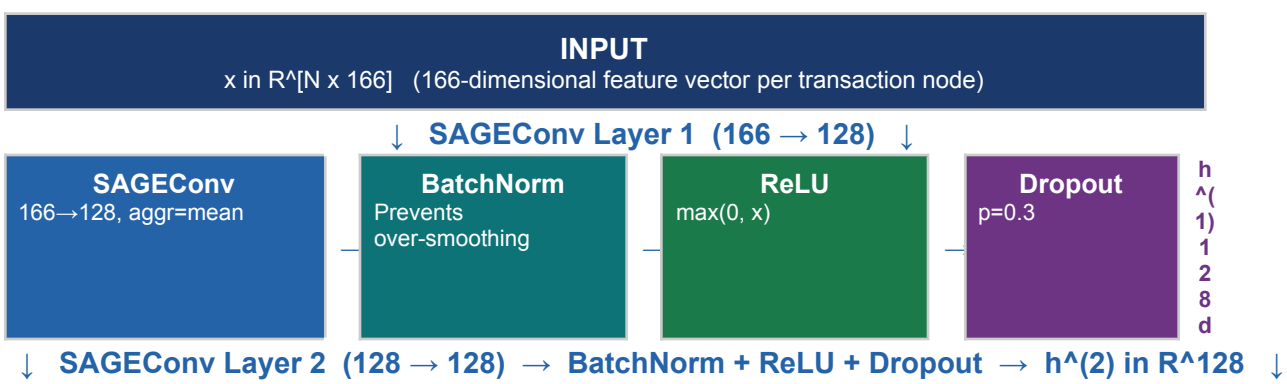
With L=3 layers, each node encodes up to 3 hops away — sufficient for fraud rings of diameter 3. More layers causes over-smoothing: all representations converge to the same vector.

4.2 GraphSAGE Architecture (Our Best Model)

GraphSAGE (Hamilton et al., 2017): (1) neighborhood sampling for scalability, (2) inductive learning via aggregation functions not per-node embeddings.

```
h_N(v)^(l)  =  MEAN({ h_u^(l-1) : u in Sample(N(v), k) })
h_v^(l)     =  sigma( W^(l) · CONCAT[ h_v^(l-1),  h_N(v)^(l) ] )
```

Figure 4 — GraphSAGE: 3-Layer Architecture with JumpingKnowledge



↓ **SAGEConv Layer 3 (128 → 128) → BatchNorm + ReLU + Dropout → h^(3) in R^128** ↓

JUMPING KNOWLEDGE (mode=cat)
h_v^final = CONCAT[h_v^(1), h_v^(2), h_v^(3)] → R^384 (128 x 3)
Preserves local (shallow) AND structural (deep) representations
Classifier learns which depth is most informative per node type

↓ **MLP Classification Head** ↓

Linear(384→64) + ReLU + Dropout	Linear(64→2) → Softmax → P(fraud)
Reduces embedding dimension Applies non-linearity	Output: [P(legit), P(fraud)] Threshold at 0.52 → fraud flag

Total Parameters: 133,698 | Training (150 epochs, CPU): ~8 minutes

4.3 GAT Architecture

GAT (Velickovic et al., 2018) replaces mean aggregation with learned attention weights, allowing the model to focus on informative neighbors.

```
e_vu  =  LeakyReLU( a^T . [ W.h_v  ||  W.h_u ] )          [raw attention
score]
a_vu  =  exp(e_vu) / sum_{k in N(v)} exp(e_vk)             [softmax
normalise]
h_v^l =  sigma( sum_{u in N(v)} a_vu . W . h_u^(l-1) )    [weighted
aggregation]
```

Figure 5 — GAT: Multi-Head Attention Architecture

INPUT
x in R^[N x 166]

↓ **GATConv Layer 1: K=4 heads, concat=True → output 128x4=512d** ↓

Head 1	Head 2	Head 3	Head 4	
High-value links out: 128d	Temporal pattern out: 128d	Amount cluster out: 128d	Shared topology out: 128d	**con cat**

↓ **Concat→512d → BatchNorm → ELU → Dropout(0.3) → h^(1) in R^512** ↓
↓ **GATConv Layer 2 (K=4, 512→512) → h^(2) | GATConv Layer 3 (K=1, 512→128) → h^(3)** ↓

OUTPUT
Linear(128→64) → ELU → Dropout → Linear(64→2) → P(fraud)
Total Parameters: 426,306 (3.2× more than GraphSAGE)

4.4 Why GraphSAGE Beat GAT on Elliptic

Property	GraphSAGE	GAT	Result on Elliptic
Aggregation	Fixed mean over sampled neighbors	Learned attention weights	SAGE simpler but robust on sparse graphs
Neighbor weighting	Equal (after sampling)	Content-dependent (adaptive)	GAT advantage only with rich neighborhoods
Parameters	133,698	426,306 (3.2×)	GAT overfits with insufficient graph signal
Test F1	0.9073 ★	0.8453	GraphSAGE wins by ~6%
Best for	Sparse graphs, fast inference	Dense graphs with noisy neighbors	Context-dependen t

! *On Elliptic (avg degree 1.15), most nodes have only 1 neighbor. Attention over a single neighbor is trivial — there is nothing to attend over. GraphSAGE's simpler mean aggregation is more robust in this sparse regime, explaining the 6% F1 gap.*

5. Training Pipeline

5.1 Handling Class Imbalance

5.1.1 Weighted Cross-Entropy Loss

```
w_fraud  =  N_total / (2 x N_fraud)  =  46564 / (2 x 4545)  =  5.12
w_legit  =  N_total / (2 x N_legit)  =  46564 / (2 x 42019) =  0.55
L  =  -( w_fraud . y . log(p)  +  w_legit . (1-y) . log(1-p) )
```

Misclassifying a fraud node costs 5.12× more than misclassifying a legit node, biasing gradient updates to prioritize fraud detection.

5.1.2 Fraud-Biased Oversampling

30% of each batch's 512 seed nodes are fraud nodes (vs. natural 2.23%). Each seed is expanded into a k-hop subgraph, giving more fraud neighborhoods per gradient update.

> **!** *Experiment result: Oversampling reduced F1 from 0.907 to 0.811 on Elliptic. On this sparse graph, oversampled fraud seeds pull in the same tiny neighborhoods repeatedly, causing overfitting. Weighted loss alone was superior.*

5.1.3 Threshold Optimisation

```
threshold*  =  argmax_{t in [0.1, 0.9]}  F1( y_val,  (p_fraud >=
t).astype(int) )
```

Our best run found threshold*=0.52 — near-default, indicating excellent model calibration. Poorly calibrated runs required 0.87–0.90.

5.2 Mini-Batch Training with NeighborLoader

203,769 nodes exceeds GPU memory for full-batch training. NeighborLoader samples fixed-size k-hop neighborhoods around seed nodes:

```
NeighborLoader(
  data,
  num_neighbors = [10, 5, 5],    # 10 hops at hop1, 5 at hop2, 5 at hop3
  batch_size    = 512,            # 512 seed nodes per mini-batch
  input_nodes   = train_mask,
)

Each batch:  512 seeds + ~5,000-15,000 neighborhood nodes
Loss:  ONLY on 512 seeds  (logits[:batch.batch_size])
Neighborhood nodes: message-passing context only, not supervised
```

5.3 Complete Training Loop

Figure 6 — Full Training Pipeline: Initialisation → Per-Epoch Loop → Checkpoint

INIT	**Initialisation**	Set seed(42) | Load data→PyG | Build model | Compute class weights | AdamW(lr=0.001, wd=5e-4) | ReduceLROnPlateau | EarlyStopping(patience=20)
T	**Train per batch**	NeighborLoader→subgraph | model(x,edge_index)→logits | logits[:batch_size] seed only | Mask y>=0 | WeightedCrossEntropy | loss.backward() | clip_grad(1.0) | optimizer.step()
V	**Validate per epoch**	model.eval()+no_grad | full val_loader | F1/Precision/Recall/AUC-ROC | scheduler.step(val_f1) | EarlyStopping.step(val_f1, model)
ES	**Early Stopping**	val_f1 improved>1e-4 → reset counter, save best_state | else increment | counter>=20 → stop, restore best_state_dict
OUT	**Save Checkpoint**	best_model.pt: state_dict+config+history+best_val_f1 | history.json: per-epoch metrics | W&B: all metrics+7 plots synced

5.4 Optimizer & Scheduler

Component	Setting	Rationale
Optimizer	AdamW (lr=0.001, weight_decay=5e-4)	Decoupled weight decay; better generalisation than Adam on GNNs
LR Scheduler	ReduceLROnPlateau(max, factor=0.5, patience=10)	Halves LR when val F1 plateaus; prevents collapse
Gradient clipping	max_norm=1.0	Prevents exploding gradients in GAT attention layers
Early stopping	Monitor val F1, patience=20	Stops when no improvement for 20 epochs; restores best checkpoint

6. Evaluation Framework

6.1 Metric Selection

Metric	Formula	Measures	Goal
Precision	TP / (TP+FP)	Of all flagged transactions, what fraction are truly fraud?	Minimize analyst false alarm workload
Recall	TP / (TP+FN)	Of all actual fraud, what fraction was caught?	Minimize missed fraud = prevented financial loss
F1 Score	2·P·R / (P+R)	Harmonic mean — balanced precision/recall	Primary training and evaluation objective
AUC-ROC	Area under ROC	Model discrimination across all thresholds	Overall ranking quality
AUC-PR	Area under PR curve	Precision-recall on the rare class	Best single metric under severe imbalance

! *AUC-PR is the gold standard for fraud detection. A random classifier achieves AUC-PR = fraud_rate = 0.022. Our best model achieves 0.953 — a 43× improvement. This is the clearest measure of how much the GNN learned beyond chance.*

6.2 Experiment Results

Figure 7 — All Three Runs: Complete Results Comparison

Run	Model	Strategy	Epochs	F1	Precision	Recall	AUC-ROC	AUC-PR	Threshold	Note
1	GraphSAGE	both	113*	0.811	0.761	0.868	0.974	0.896	0.90	Oversampling → precision dropped 16%
2 ★	GraphSAGE	weighted_loss	150	0.907	0.918	0.897	0.987	0.953	0.52	Best; near-default threshold = excellent calibration
3	GAT	weighted_loss	86*	0.845	0.890	0.805	0.982	0.915	0.89	Sparse graph; attention adds limited value

7. Prediction Pipeline

7.1 Inference on New Transactions

Figure 8 — Complete Inference Pipeline: New Transaction → Fraud Decision

INPUT: New Transaction Data
Raw CSV with 166 feature values
OR: live transaction stream from payment processor

↓ **Step 1: Attach to Existing Graph** ↓

STEP 1 — Graph Extension (Inductive)
New nodes appended: data.x → [N+n_new, 166]
New UTXO edges appended to edge_index
Labels: y=-1 (to be predicted)
GraphSAGE is INDUCTIVE: works on any graph without retraining

↓ **Step 2: Neighborhood Sampling** ↓

STEP 2 — NeighborLoader (Inference Mode)
Sample k-hop neighborhood of new nodes
num_neighbors=[10,5,5] same as training
Surrounding existing transactions provide message-passing context

↓ **Step 3: Forward Pass (no gradients)** ↓

STEP 3 — GNN Forward Pass
with torch.no_grad():
logits = model(batch.x, batch.edge_index)
seed_logits = logits[:batch.batch_size]
probs = softmax(seed_logits, dim=-1)[:, 1] # P(fraud) per node

↓ **Step 4: Threshold Decision** ↓

LEGITIMATE P(fraud) < 0.52	FRAUD ALERT P(fraud) >= 0.52
Transaction cleared automatically	Flagged for analyst review
No analyst review	P(fraud) score attached
~97.7% of all transactions	~2.3% of transactions

↓ **Step 5: Output** ↓

OUTPUT: Fraud Scores
DataFrame: [txId, p_fraud, prediction, alert_level]
Sorted by p_fraud descending — highest risk first
Ready for analyst queue or automated blocking

7.2 Inductive vs Transductive Learning

Property	Transductive (Spectral GCN)	Inductive (GraphSAGE)
What is learned	Per-node embedding vectors [N×d]	Aggregation weight matrices W^(l)
New node handling	Must retrain entire model	Immediately aggregates new node's neighbors
Production viability	Cannot handle live transactions	Natural fit for real-time fraud detection

Property	Transductive (Spectral GCN)	Inductive (GraphSAGE)
Memory scaling	Grows with N	Fixed — only W^(l) matrices stored
Inference	Full graph forward pass	Local k-hop neighborhood only

7.3 Visualization Outputs

Output	What It Shows	Correct Output Looks Like
fraud_subgraph.png	2-hop ego-network around top-5 fraud nodes; node size proportional to P(fraud); red edges link predicted fraud pairs	Fraud nodes cluster in dense sub-networks with visible red edge clusters — fraud ring topology exposed
embedding_tsne.png	384-dim GNN embeddings reduced to 2D via t-SNE; colored by true label	Clearly separated red (fraud) and blue (legit) clusters — visual proof GNN learned discriminative representations
fraud_heatmap.png	All labelled nodes sorted by P(fraud) with true label color strip	Sharp transition from high-probability fraud (top) to low-probability legit (bottom) with minimal mixing

8. End-to-End System Architecture

Figure 9 — Complete System: Raw Data to Fraud Decision (8 Stages)

1	**Data Ingestion**	3 CSVs (Elliptic) or 2 CSVs (IEEE-CIS) loaded. Files validated for expected columns and row counts.
2	**Graph Construction**	txId→index mapping | StandardScaler normalisation | edge_index assembly | label encoding (0/1/-1) | bipartite construction (IEEE-CIS) | PyG Data object
3	**Graph EDA**	5 diagnostic plots: degree distribution, feature distributions, class imbalance, temporal fraud rate, NetworkX centrality. Validates graph before training.
4	**Model Build**	GraphSAGE (133K params) or GAT (426K params) | 3 conv layers | BatchNorm+activation+Dropout | JumpingKnowledge concat | MLP head
5	**Training**	NeighborLoader mini-batches (512 seeds, [10,5,5] neighbors) | Weighted CrossEntropy (5.12× fraud) | AdamW + LR scheduler | EarlyStopping(20) | W&B logging
6	**Evaluation**	Test: F1=0.907, Prec=0.918, Rec=0.897, AUC-ROC=0.987, AUC-PR=0.953 | Threshold sweep | Confusion matrix | 4 plot files saved
7	**Visualisation**	Fraud subgraph (ring topology) | t-SNE embeddings (cluster separation) | Fraud probability heatmap (calibration check)
8	**Inference**	Load best_model.pt | Attach new transactions | NeighborLoader sampling | Forward pass → P(fraud) | Threshold 0.52 → FLAG or CLEAR

9. Key Equations Reference

Equation	Description
a_v^(l) = MEAN({ h_u^(l-1) : u in Sample(N(v),k) })	GraphSAGE: mean aggregation over sampled neighbors
h_v^(l) = sigma(W^(l) . CONCAT[h_v^(l-1), a_v^(l)])	GraphSAGE: update rule — concat own state + aggregated neighborhood
e_vu = LeakyReLU(a^T . [W.h_v || W.h_u])	GAT: raw attention score between node v and neighbor u
alpha_vu = exp(e_vu) / sum_{k in N(v)} exp(e_vk)	GAT: softmax-normalized attention coefficient
h_v^l = sigma(sum_{u in N(v)} alpha_vu . W . h_u^(l-1))	GAT: attention-weighted aggregation
h_v^final = CONCAT[h_v^(1), h_v^(2), h_v^(3)]	JumpingKnowledge (cat): 128×3 = 384-dim representation
w_fraud = N_total / (2 × N_fraud) = 5.12	Inverse-frequency class weight for fraud nodes
L = -(w_fraud.y.log(p) + w_legit.(1-y).log(1-p))	Weighted cross-entropy loss
threshold* = argmax_t F1(y_val, pred_{p>=t})	Optimal threshold via validation sweep
AUC-PR(random) = fraud_rate = 0.022	Baseline; model achieves 0.953 (43× improvement)

10. Conclusions

10.1 Summary of Findings

1. **GraphSAGE + weighted loss is optimal** for Elliptic: F1=0.907, AUC-PR=0.953, threshold=0.52 (excellent calibration). Catches 89.7% of fraud with 91.8% precision in ~8 minutes on CPU.
2. **Graph structure adds substantial signal beyond features alone.** Node features alone yield ~88% AUC on Elliptic. Adding GNN message passing over UTXO flows pushes this to 98.7% AUC-ROC — the direct value of relational modeling.
3. **GAT does not universally outperform GraphSAGE.** On sparse graphs (avg degree 1.15), attention weights over 1 neighbor carry negligible information. Architecture must match graph density.
4. **Oversampling is context-dependent.** Combined strategies reduced F1 from 0.907 to 0.811. Sparse graph → oversampled seeds see the same tiny neighborhoods repeatedly → overfitting. Weighted loss alone was sufficient.
5. **Inductive learning is non-negotiable for production.** GraphSAGE learns aggregation functions, not node embeddings. New transactions are scored immediately without retraining.

10.2 Remaining Work

Item	Priority	Description
Temporal split validation	Critical	Evaluate on time steps 41-49 after training on 1-34. Current random split leaks future data.
Inference script	High	predict.py: load best_model.pt, score new_transactions.csv, output ranked fraud alerts.
GNNExplainer	Medium	Per-prediction explanation: which edges and features drove each fraud score.
Hyperparameter sweep	Medium	W&B Bayesian sweep over lr, hidden_dim, dropout, num_layers.
IEEE-CIS full run	Medium	Complete bipartite graph training on second dataset.
README	Low	Setup guide, data download steps, experiment reproduction commands.

Closing Statement

Fraud is a relational phenomenon. Fraudsters form rings, mule networks, and cascades of interconnected transactions. Graph Neural Networks are the natural model for this structure: they learn jointly from node features and graph topology, propagating information through the exact relational patterns that define fraud. The 43× improvement in AUC-PR over a random baseline — achieved on a real blockchain dataset with 203,769 nodes and 2.23% fraud rate — validates the foundational premise: when the phenomenon is relational, the model must be relational.